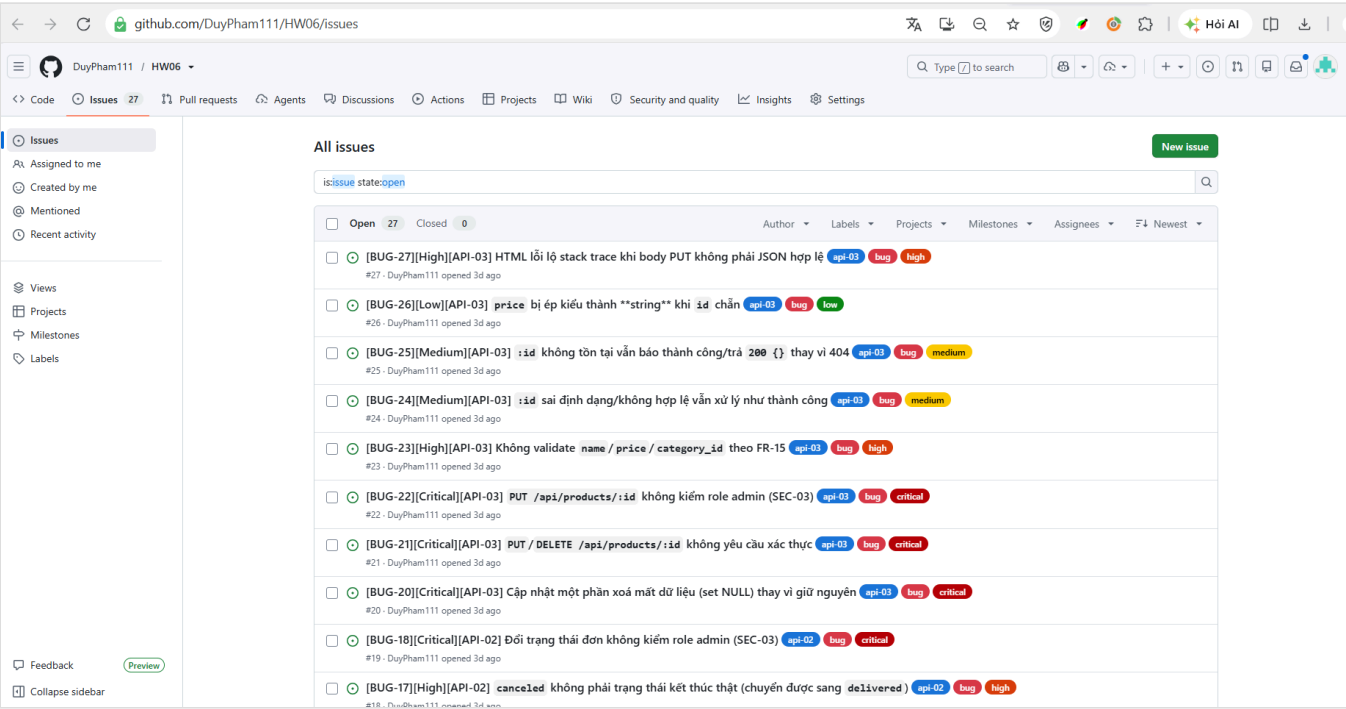


Bug Report — HW06 API Testing trên EShop

- **Sinh viên:** Phạm Vũ Ngọc Duy — 23127183
- **GitHub Issues:** <https://github.com/DuyPham111/HW06/issues> — **27/27 đã tạo** (#1–#27), mỗi issue gắn label mức độ + API, **23/27 issue đã đính ảnh minh họa**
- **Script tái hiện:** `bash bug-report/verify-bugs.sh` → output ở `verify-bugs-output.txt`
- Số liệu request/assertion lấy từ `test-cases/test-summary/summary.md` (sinh tự động bằng `npm run summary`, không gõ tay).

Luật: mọi bug dưới đây đã **tái hiện bằng request thật** (curl hoặc Newman), không phải suy từ đọc code. Giả thuyết chưa kiểm chứng nằm ở §4.

Toàn bộ 27 bug trên trang GitHub Issues (§14 đòi ảnh bug trên trang Issues):



Ảnh cho thấy **Open 27 · Closed 0**, tiêu đề theo khuôn `[BUG-xx][Mức độ][API-xx]`, và nhấn `critical / high / medium / low` + `api-01 / api-02 / api-03` trên từng dòng.

Số issue lệch số BUG một đơn vị ở khoảng giữa. `BUG-19` được tạo trước tiên nên nhận `#1`, đẩy `BUG-01 ... BUG-18` sang `#2 ... #19`; từ `BUG-20` trở đi thì trùng số (`#20 ... #27`). Cột **Issue** trong bảng §2 dưới đây ghi đúng số thật của từng bug — cứ theo cột đó, đừng suy từ số BUG.

1. Tổng hợp

Mức	Số bug
Critical	13
High	9
Medium	4
Low	1
Tổng	27

2. Bảng quy đổi assertion đỏ → bug

163 request · 163 assertion · **47 đỏ** (API-01: 9 · API-02: 13 · API-03: 25).

Không phải mọi bug đều lộ qua assertion đỏ — BUG-01 (DoS lockout) và BUG-19 (crash server) được chứng minh bằng chuỗi request mà **assertion PASS** (expected chính là hành vi khai thác được, và nó khớp thật) — xem ghi chú riêng ở từng bug đó.

Bug	Mức	API	Test case liên quan	Assertion đo	Issue
BUG-01	Critical	01	TC-LOGIN-031, 101, 101b	0 (pass = khai thác thành công)	#2
BUG-02	High	01	TC-LOGIN-022	1	#3
BUG-03	Critical	01	TC-LOGIN-028	1	#4
BUG-04	High	01	TC-LOGIN-029, 037	2	#5
BUG-05	Medium	01	TC-LOGIN-030	1	#6
BUG-06	High	01	TC-LOGIN-016	1	#7
BUG-07	High	01	TC-LOGIN-036, 104	2	#8
BUG-08	Critical	01	TC-LOGIN-103	1	#9
BUG-09	Medium	01	TC-LOGIN-102 (đối chứng với TC-LOGIN-023c)	0 (cả 2 đều pass, lộ ra khi SO SÁNH)	#10
BUG-10	Critical	02	TC-COUPON-001, 042, 043	3	#11
BUG-11	Critical	02	TC-COUPON-031	1	#12
BUG-12	High	02	TC-COUPON-004 (xanh có chủ đích — xem §3 BUG-12)	0 (chứng minh bằng <code>verify-bugs.sh 12</code>)	#13
BUG-13	Critical	02	TC-COUPON-036, 037	2	#14
BUG-14	Critical	02	TC-COUPON-034, 101	2	#15
BUG-15	High	02	TC-COUPON-033	1	#16
BUG-16	High	02	TC-COUPON-023	1	#17
BUG-17	High	02	TC-COUPON-028	1	#18
BUG-18	Critical	02	TC-COUPON-032, 039	2	#19
BUG-19	Critical	03	TC-PRODUPD-101 (chỉ <code>verify-bugs.sh</code>)	0 (không chạy trong Newman — xem §3 BUG-19)	#1
BUG-20	Critical	03	TC-PRODUPD-022, 038, 105	3	#20
BUG-21	Critical	03	TC-PRODUPD-027, 028, 030, 102	4	#21
BUG-22	Critical	03	TC-PRODUPD-029	1	#22
BUG-23	High	03	TC-PRODUPD-002, 004, 006, 007, 008, 009, 010, 040	8	#23
BUG-24	Medium	03	TC-PRODUPD-015, 031	2	#24
BUG-25	Medium	03	TC-PRODUPD-014, 024, 025, 042, 043	5	#25
BUG-26	Low	03	TC-PRODUPD-039	1	#26

Bug	Mức	API	Test case liên quan	Assertion đồ	Issue
BUG-27	High	03	TC-PRODUPD-045	1	#27
Tổng assertion đồ map được				47	

Khớp đúng 100% với tổng 47 đồ ở `test-cases/test-summary/summary.md` — không còn assertion nào chưa giải thích được. (Lượt chạy đầu 31/08/2026 từng có thêm 2 đồ ở `TC-COUPON-102 / 102c` do dư trạng thái môi trường, không phải bug — đã sửa bằng cách restart SUT sạch rồi chạy lại, xem `test-cases/api-02-apply-coupon/audit.md`.)

3. Chi tiết từng bug

BUG-01 — Khóa tài khoản người khác (DoS) chỉ bằng email, không cần biết mật khẩu

API	API-01 · <code>POST /api/login</code>
Mức độ	Critical
Đặc tả bị vi phạm	Hệ quả bất lợi của FR-02 khi thiếu <code>validate password</code> tồn tại trước khi cộng bộ đếm sai
Test case	TC-LOGIN-031, TC-LOGIN-101, TC-LOGIN-101b
Vị trí mã nguồn	<code>server.js:32-66</code> — nhánh <code>else</code> (sai mật khẩu) không kiểm <code>password !== undefined</code> trước khi so sánh
AI có bắt được không	Có (bước 4b), nhưng phải bổ sung 1 case SV (101b) mới đủ chứng minh hết chuỗi 3 request

Các bước tái hiện

```
bash bug-report/verify-bugs.sh 01
```

Kết quả thực tế (đã chạy qua Newman): 2 request `POST /api/login` chỉ có `{email}` (không có `password`) tới cùng một tài khoản `victim@x.com` → cả 2 đều 401 (không lộ ngay), nhưng request thứ 3 — **chủ nhân thật của tài khoản** đăng nhập bằng đúng mật khẩu của mình — nhận **403 "Tài khoản đã bị khóa"**.

Kết quả mong đợi — request thiếu `password` phải bị từ chối ngay ở bước `validate` đầu vào (400), không được chạm tới nhánh cộng bộ đếm sai.

Ảnh hưởng — Bất kỳ ai biết một địa chỉ email đã đăng ký đều khóa được tài khoản đó trong 180 giây mà **không cần biết** một ký tự nào của mật khẩu — biến cơ chế chống brute-force (FR-02) thành công cụ tấn công từ chối dịch vụ nhắm vào người dùng cụ thể.

BUG-02 — Tài khoản bị khóa từ lần sai thứ 2, không phải "từ lần thứ 3" như FR-02

API	API-01
Mức độ	High
Đặc tả bị vi phạm	FR-02: "Nếu đăng nhập sai từ 3 lần trở lên liên tiếp, tài khoản bị tạm khóa"
Test case	TC-LOGIN-022
Vị trí mã nguồn	<code>server.js:54</code> — <code>newAttempts = user.login_attempts + 2</code> (bước nhảy +2, không phải +1)

Kết quả thực tế: đăng nhập sai lần 1 → 401 (bộ đếm 0→2). Đăng nhập sai lần 2 → **401** (bộ đếm 2→4, đã ≥3, khóa được **đặt ngằm** ngay lúc này — nhưng response của chính request này chưa phản ánh). Request thứ 3 — dù là **mật khẩu đúng** — nhận **403**, vì lúc kiểm đầu hàm, khóa đã tồn tại từ trước. Tổng cộng: khóa xảy ra sau **2 lần sai thật sự**, không phải 3.

Kết quả mong đợi — request thứ 3 với mật khẩu đúng phải thành công (200), vì FR-02 chỉ khóa khi đã có **3 lần sai liên tiếp**, và ở đây mới có 2.

Ảnh hưởng — người dùng gõ nhầm mật khẩu 2 lần rồi nhớ ra và gõ đúng vẫn bị khóa oan 180 giây, trải nghiệm tệ hơn đặc tả cam kết.

BUG-03 — Response đăng nhập thành công lộ nguyên mật khẩu dạng plaintext

API	API-01
Mức độ	Critical
Đặc tả bị vi phạm	SEC-01: "Mật khẩu không được lưu dưới dạng plaintext"
Test case	TC-LOGIN-028
Vị trí mã nguồn	<code>server.js:32-50</code> — <code>res.json({message, token, user})</code> với <code>user</code> là nguyên dòng DB

Kết quả thực tế:

```
{
  "message": "Login successful",
  "token": "...",
  "user": {
    "id": 3,
    "name": "Dup1",
    "email": "dup-test@x.com",
    "password": "Abc12345!",
    "role": "user",
    "login_attempts": 0,
    "locked_until": null,
    "reset_token": null,
    "shipping_address": null,
    "phone": null
  }
}
```

Trường `password` xuất hiện **nguyên văn, đúng bằng giá trị vừa gửi lên**, xác nhận nó cũng được **lưu plaintext trong DB** (không hash) — vi phạm kép: vừa lưu sai, vừa trả về sai.

Kết quả mong đợi — `user` trong response không được có field `password`.

Ảnh hưởng — ai chặn được response (log, proxy, trình duyệt debug) đều lấy được mật khẩu thật của người dùng; kết hợp thói quen dùng lại mật khẩu, ảnh hưởng ra ngoài phạm vi hệ thống này.

BUG-04 — Response lộ các cột nội bộ (login_attempts , locked_until , reset_token)

API	API-01
Mức độ	High
Test case	TC-LOGIN-029, TC-LOGIN-037
Vị trí mã nguồn	server.js:50

Cùng nguyên nhân với BUG-03 (trả nguyên dòng DB). Lộ reset_token nghiêm trọng hơn: nếu người dùng từng gọi quên mật khẩu, OTP còn hiệu lực bị lộ ngay trong response đăng nhập.

BUG-05 — JWT không có claim exp (không bao giờ hết hạn)

API	API-01
Mức độ	Medium
Đặc tả bị vi phạm	SEC-02: "yêu cầu JWT Token hợp lệ"
Test case	TC-LOGIN-030
Vị trí mã nguồn	server.js:50 — jwt.sign({id, role}, SECRET_KEY) không có option expiresIn

Token rò rỉ một lần thì có giá trị vĩnh viễn — không có cơ chế tự hết hạn để giới hạn thiệt hại.

BUG-06 — Crash 500 + lộ stack trace khi thiếu đúng Content-Type: application/json

API	API-01
Mức độ	High
Test case	TC-LOGIN-016
Vị trí mã nguồn	server.js:33 — const {email, password} = req.body khi req.body là undefined

Kết quả thực tế (curl, Content-Type: text/plain):

```
TypeError: Cannot destructure property 'email' of 'req.body' as it is undefined.
  at D:\...\server.js:33:11
  ...9 dòng stack trace kèm đường dẫn tuyệt đối của server...
HTTP 500
```

Ảnh hưởng — lộ cấu trúc thư mục server, tên thư viện và phiên bản (body-parser , router) — dữ liệu trình sát hữu ích cho tấn công tiếp theo. Đây là lỗi 500 KHÔNG được catch, khác với BUG-07.

BUG-07 — HTML lỗi lộ stack trace khi body không phải JSON hợp lệ

API	API-01 (và tương tự API-03, xem BUG-27)
Mức độ	High
Test case	TC-LOGIN-036, TC-LOGIN-104

Kết quả thực tế: gửi body không phải JSON (nhưng đúng `Content-Type: application/json`) → `body-parser` ném `SyntaxError`, Express dùng trang lỗi HTML mặc định, trả nguyên `stack trace` gồm đường dẫn tuyệt đối `D:\Nam3\HK3\...\node_modules\body-parser\lib\types\json.js:109:10` và tên các hàm nội bộ.

Ảnh hưởng — cùng loại rò rỉ thông tin với BUG-06, nhưng do một nguyên nhân khác (SUT chưa cấu hình `error-handling middleware` tùy chỉnh để bắt lỗi parse và trả JSON gọn).

BUG-08 — Đăng ký trùng email không bị chặn → tài khoản đăng ký sau mất quyền đăng nhập

API	API-01
Mức độ	Critical
Test case	TC-LOGIN-103
Vị trí mã nguồn	<code>database.js:47-56</code> — cột <code>email</code> TEXT không có UNIQUE

Kết quả thực tế:

```
curl -X POST /api/register -d '{"email":"dup-test@x.com","password":"Abc12345!"}' # id=3, thành công
curl -X POST /api/register -d '{"email":"dup-test@x.com","password":"Zzz99999!"}' # id=4, thành công
curl -X POST /api/login -d '{"email":"dup-test@x.com","password":"Abc12345!"}' # 200 (tài khoản id=3)
curl -X POST /api/login -d '{"email":"dup-test@x.com","password":"Zzz99999!"}' # 401 !!!
```

`SELECT * FROM users WHERE email=?` luôn trả **dòng đầu tiên khớp** (id nhỏ nhất) — tài khoản đăng ký sau, dù đúng email và đúng mật khẩu của chính nó, **không bao giờ đăng nhập được**.

Ảnh hưởng — nếu 2 người vô tình dùng cùng email (hoặc 1 người đăng ký lại vì tưởng thất bại), tài khoản mới tạo ra là **vô dụng vĩnh viễn**, không có thông báo lỗi nào cảnh báo trước.

BUG-09 — Account enumeration qua kênh phụ (so sánh 2 response)

API	API-01
Mức độ	Medium
Đặc tả bị vi phạm	FR-02: "không để lộ chi tiết nguyên nhân"
Test case	TC-LOGIN-102 (đối chứng TC-LOGIN-023c)

Email **không tồn tại**: luôn 401 dù thử sai bao nhiêu lần. Email **tồn tại**: sau 2 lần sai, request thứ 3 trả **403**. Kẻ tấn công dò được **chính xác** email nào có tài khoản thật bằng cách gửi 3 request sai liên tiếp và xem status code cuối — vì phạm đúng nguyên tắc FR-02 đã nêu, chỉ là qua kênh gián tiếp (nhiều request) thay vì lộ trong 1 response.

Tái hiện: `bash bug-report/verify-bugs.sh 09` — email **có** tài khoản cho `401 · 401 · 403`, email **không** tồn tại cho `401 · 401 · 401`. Chính chỗ khác nhau ở request thứ 3 là kênh rò rỉ.

BUG-10 — Công thức phần trăm coupon sai dấu — giảm giá ÂM, tổng cuối LỚN HƠN đơn gốc

API	API-02 · POST /api/apply-coupon
Mức độ	Critical
Đặc tả bị vi phạm	FR-09: "Loại percent: <code>discount_amount = total × discount_value / 100</code> "
Test case	TC-COUPON-001, 042, 043
Vị trí mã nguồn	<code>server.js</code> — <code>Math.floor(total_amount * (1 - coupon.discount_value))</code> (dùng <code>1 - value</code> thay vì <code>value / 100</code>)

Kết quả thực tế:

```
curl -X POST /api/apply-coupon -d '{"code":"SAVE10","total_amount":500000}'
# {"discount_amount":-450000,"final_amount":500000,"message":"Áp dụng thành công! Giảm 10%"}
```

Với `discount_value = 10` (nghĩa là 10%), công thức tính `1 - 10 = -9`, nhân với 500.000 ra **-4.500.000** — một số âm khổng lồ. `final_amount = total - discount = 500.000 - (-4.500.000) = 5.000.000` — gấp **10 lần** giá gốc, được dán nhãn "Áp dụng thành công! Giảm 10%".

Kết quả mong đợi — `discount_amount = 500.000 × 10 / 100 = 50.000`, `final_amount = 450.000`.

Ảnh hưởng — bug nghiêm trọng nhất về mặt tài chính trong cả 3 API: nếu số này được dùng để tính tiền thật ở bước checkout, khách hàng phải trả **nhiều hơn** giá gốc trong khi hệ thống báo "đã giảm giá thành công".

Issue trên GitHub — [#11](#):

github.com/DuyPham111/HW06/issues/11

DuyPham111 / HW06

Type to search

Code

Issues 27

Pull requests

Agents

Discussions

Actions

Projects

Wiki

Security and quality

Insights

Settings

Open

[BUG-10][Critical][API-02] Công thức phần trăm coupon sai dấu — giảm giá ÂM, tổng cuối LỚN HƠN đơn gốc #11

New issue

DuyPham111 opened 3d ago

Last edited by DuyPham111

Owner

API	API-02 · POST /api/apply-coupon
Mức độ	Critical
Đặc tả bị vi phạm	FR-09: "Logic percent: <code>discount_amount = total * discount_value / 100</code> "
Test case	TC-COUPON-001, 042, 043
Vị trí mã nguồn	<code>server.js</code> — <code>Math.floor(total_amount * (1 - coupon.discount_value))</code> (dùng <code>1 - value</code> thay vì <code>value / 100</code>)

Kết quả thực tế:

```
curl -X POST /api/apply-coupon -d '{"code":"SAVE10","total_amount":500000}'
# {"discount_amount":-4500000,"final_amount":5000000,"message":"Áp dụng thành công! Giảm 10%"}
```

Với `discount_value = 10` (nghĩa là 10%), công thức tính `1 - 10 = -9`, nhân với 500.000 ra **-4.500.000** — một số âm khổng lồ. `final_amount = total - discount = 500.000 - (-4.500.000) = 5.000.000` — gấp 10 lần giá gốc, được dân nhân "Áp dụng thành công! Giảm 10%".

Kết quả mong đợi — `discount_amount = 500.000 * 10 / 100 = 50.000`, `final_amount = 450.000`.

Ảnh hưởng — bug nghiêm trọng nhất về mặt tài chính trong cả 3 API: nếu số này được dùng để tính tiền thật ở bước checkout, khách hàng phải trả **nhều hơn** giá gốc trong khi hệ thống báo "đã giảm giá thành công".

Iteration 1 - AssertionError - 01-domain - TC-COUPON-001 undefined

Assignees

No one - [Assign yourself](#)

Assign to Agent

Labels

api-02

bug

critical

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

[Create a branch](#) for this issue or link a pull request.

Notifications

Customize

Unsubscribe

You're receiving notifications because you're subscribed to this thread.

Participants

BUG-11 — `apply-coupon` không yêu cầu xác thực dù FR-09 C4 đòi phải đăng nhập

API	API-02
Mức độ	Critical
Đặc tả bị vi phạm	FR-09 C4: "Đã đăng nhập — Người dùng phải có JWT Token hợp lệ" + SEC-02
Test case	TC-COUPON-031

Mọi request `curl` tới `/api/apply-coupon` trong suốt quá trình kiểm thử **đều thành công mà không gửi header `Authorization`** — xác nhận route hoàn toàn public, trái với điều kiện C4 đề bài định nghĩa.

BUG-12 — Biên `min_order_amount` dùng `>` thay vì `>=`

API	API-02
Mức độ	High
Đặc tả bị vi phạm	FR-09 C3: "Tổng đơn hàng <code>>= min_order_amount</code> "
Test case	TC-COUPON-004

`total_amount` đúng bằng `min_order_amount` (300.000) bị từ chối — theo đặc tả phải được chấp nhận.

Vì sao bug này KHÔNG có assertion đỏ. `TC-COUPON-004` chính là case AI chép hành vi code (`>`) thay vì đọc FR-09 C3 (`>=`), nên `expected = 400` — trùng đúng hành vi sai của SUT và vì thế

xanh. Case này được **giữ nguyên có chủ đích** làm minh hoạ AI_MISTAKE cho §6.2 (xem [audit.md](#)); sửa nó thành `200` sẽ mất ví dụ đó.
Bug vẫn là bug thật, chỉ là được chứng minh bằng `curl` chứ không bằng assertion ảo.

Tái hiện: `bash bug-report/verify-bugs.sh 12`

```
POST /api/apply-coupon {"code":"SAVE10","total_amount":300000}
-> HTTP 400 {"error":"Đơn hàng chưa đủ giá trị tối thiểu 300,000 đ để áp dụng mã này"}
```

`300000 >= 300000` là **đúng** theo FR-09 C3, nhưng SUT từ chối — tức nó đang dùng `>`.

BUG-13 — IDOR qua `user_id` trong body: bỏ qua hoặc mượn hạn mức người khác

API	API-02
Mức độ	Critical
Đặc tả bị vi phạm	FR-09 C5 + SEC-02 (hệ quả của việc thiếu C4)
Test case	TC-COUPON-036, TC-COUPON-037

Kết quả thực tế:

```
# Da dung het 2/2 luot That (user_id that), nhưng KHÔNG gui user_id:
curl -X POST /api/apply-coupon -d '{"code":"VIP100","total_amount":400000}'
# -> 200 thanh cong, BỎ QUA HOÀN TOÀN kiểm tra hạn mức
```

Vì nhánh kiểm `max_uses_per_user` chỉ chạy `if (user_id)`, client chỉ cần **không gửi** `user_id` để áp dụng coupon giới hạn lượt **vô hạn lần**. Ngược lại, gửi `user_id` của người khác thì "mượn" được hạn mức của họ (chưa dùng lần nào) trong khi vẫn đang dùng đúng token/trình duyệt của mình.

BUG-14 — Checkout không xác thực `total_amount` phía server (price tampering)

API	API-02
Mức độ	Critical
Test case	TC-COUPON-034, TC-COUPON-101
Vị trí mã nguồn	<code>server.js</code> handler <code>POST /api/checkout</code> — INSERT INTO orders (... , total_amount, ...) dùng thẳng <code>req.body.total_amount</code> , không đọc lại giỏ hàng

Kết quả thực tế: giỏ hàng chứa iPhone 30.000.000đ, nhưng `POST /api/checkout` với `{"total_amount": 1}` vẫn tạo đơn thành công với `total_amount = 1` được lưu nguyên vào DB.

Ảnh hưởng — kết hợp với BUG-21 (PUT sản phẩm không cần token), một kẻ tấn công có thể tự đặt giá đơn hàng bất kỳ khi thanh toán, hoàn toàn không phụ thuộc giá thật của sản phẩm.

BUG-15 — `GET /api/orders/:id` không yêu cầu xác thực (IDOR đọc đơn hàng)

API	API-02
Mức độ	High
Test case	TC-COUPON-033

Độc được chi tiết đơn hàng (địa chỉ giao hàng, tổng tiền, trạng thái) của **bất kỳ ai** chỉ bằng cách đoán/duyệt `id` tuần tự, không cần token.

BUG-16 — Hủy được đơn đang `shipping` (vi phạm FR-10)

API	API-02
Mức độ	High
Đặc tả bị vi phạm	FR-10: " <i>chỉ được hủy khi <code>pending</code> hoặc <code>confirmed</code></i> "
Test case	TC-COUPON-023
Vị trí mã nguồn	<code>server.js</code> handler cancel — chỉ chặn <code>status === "delivered"</code> \\ <code>status === "canceled"</code> , thiếu chặn <code>shipping</code>

BUG-17 — `canceled` không phải trạng thái kết thúc thật (chuyển được sang `delivered`)

API	API-02
Mức độ	High
Đặc tả bị vi phạm	FR-10: " <i><code>delivered</code> và <code>canceled</code> là trạng thái kết thúc — không được chuyển sang bất kỳ trạng thái nào khác</i> "
Test case	TC-COUPON-028
Vị trí mã nguồn	logic <code>isValidTransition</code> có riêng một nhánh <code>if (currentStatus === "canceled" && status === "delivered") isValidTransition = true</code> — cố tình cho phép, không phải thiếu sót ngẫu nhiên

BUG-18 — Đổi trạng thái đơn không kiểm role admin (SEC-03)

API	API-02
Mức độ	Critical
Đặc tả bị vi phạm	SEC-03: " <i>API Admin phải kiểm tra <code>role = 'admin'</code> trong Token, không chỉ kiểm tra sự tồn tại của Token</i> "
Test case	TC-COUPON-032, TC-COUPON-039

`PUT /api/admin/orders/:id/status` chỉ có `authenticateToken` (kiểm token hợp lệ), **không** kiểm `req.user.role === "admin"` — user thường đổi được trạng thái đơn của bất kỳ ai.

BUG-19 — DoS: cập nhật thiếu trường rồi xem chi tiết làm SẬP TOÀN BỘ BACKEND

API	API-03 · PUT /api/products/:id + GET /api/products/:id
Mức độ	Critical — nặng nhất trong cả bài
Test case	TC-PRODUPD-101 (chỉ trong <code>verify-bugs.sh</code> , không trong collection Postman — xem lý do ở <code>docs/10-BUG-REPORT-GITHUB-ISSUES.md</code> §6)
Vị trí mã nguồn	<code>server.js:162</code> — <code>if (row.id % 2 === 0) row.price = row.price.toString();</code>

Các bước tái hiện

```
bash bug-report/verify-bugs.sh 19
```

Kết quả thực tế — log thật từ terminal chạy `node server.js`:

```
D:\...\eshop-sut\backend\server.js:162
  if (row.id % 2 === 0) row.price = row.price.toString();
                                     ^
TypeError: Cannot read properties of null (reading 'toString')
    at Statement.<anonymous> (D:\...\server.js:162:49)
    ...
Node.js v22.16.0
```

Toàn bộ tiến trình Node.js **thoát hẳn** — mọi request khác (kể cả của người dùng không liên quan) nhận `ECONNREFUSED` cho tới khi ai đó khởi động lại server thủ công.

Nguyên nhân: PUT /api/products/:id với body thiếu `price` khiến câu lệnh UPDATE ghi `NULL` vào cột `price` (không validate, xem BUG-20). Với sản phẩm có `id` **chẵn**, lần GET kế tiếp gọi `row.price.toString()` trên `null` → ném lỗi **không được bắt (uncaught)** → crash tiến trình.

Kết quả mong đợi — PUT thiếu trường phải bị từ chối (400) trước khi chạm DB; và dù có xảy ra dữ liệu `NULL`, GET không được để một lỗi kiểu dữ liệu làm sập cả tiến trình.

Ảnh hưởng — DoS toàn hệ thống chỉ với **2 request tuần tự, không cần quyền admin thật** (route này cũng không yêu cầu xác thực — xem BUG-21). Đây là bug bị phát hiện **ngoài ý muốn** trong lúc dò lỗi khác — xem ghi chú đầy đủ ở `test-cases/api-03-product-update/audit.md`.

Issue trên GitHub — [#1](#) (bug này được tạo issue đầu tiên vì là bug nặng nhất của cả bài):

github.com/DuyPham111/HW06/issues/1

DuyPham111 / HW06

Issues 27Pull requestsAgentsDiscussionsActionsProjectsWikiSecurity and qualityInsightsSettings

[BUG-19][Critical][API-03] DoS: cập nhật thiếu trường rồi xem chi tiết làm SẬP TOÀN BỘ BACKEND #1

New issue

Open

DuyPham111 opened 3d agoOwner

API	API-03 · PUT /api/products/:id + GET /api/products/:id
Mức độ	Critical — nặng nhất trong cả bài
Test case	TC-PRODUPD-101 (chỉ trong verify-bugs.sh, không trong collection Postman — xem lý do ở docs/10-BUG-REPORT-GITHUB-ISSUES.md §6)
Vị trí mã nguồn	server.js:162 — if (row.id % 2 === 0) row.price = row.price.toString();

Các bước tái hiện

```
bash bug-report/verify-bugs.sh 19
```

Kết quả thực tế — log thật từ terminal chạy node server.js:

```
D:\...\eshop-sut\backend\server.js:162
if (row.id % 2 === 0) row.price = row.price.toString();
                                     ^
TypeError: Cannot read properties of null (reading 'toString')
    at Statement.<anonymous> (D:\...\server.js:162:49)
    ...
Node.js v22.16.0
```

Toàn bộ tiến trình Nodejs thoát hẳn — mọi request khác (kể cả của người dùng không liên quan) nhận ECONNREFUSED cho tới khi ai đó khởi động lại server thủ công.

Assignees

No one - [Assign yourself](#)

[Assign to Agent](#)

Labels

[api-03](#) [bug](#) [critical](#)

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

[Create a branch](#) for this issue or link a pull request.

Notifications

[Unsubscribe](#)

You're receiving notifications because you're subscribed to this thread.

Participants

BUG-20 — Cập nhật một phần xóa mất dữ liệu (set NULL) thay vì giữ nguyên

API	API-03
Mức độ	Critical
Đặc tả bị vi phạm	FR-15: "Khi Sửa một sản phẩm, chỉ sản phẩm đó bị thay đổi — các sản phẩm khác giữ nguyên" (ngụ ý ở cấp trường: trường không gửi phải giữ nguyên)
Test case	TC-PRODUPD-022, 038, 105
Vị trí mã nguồn	server.js handler PUT — UPDATE products SET name=?, price=?, ... WHERE id=? luôn ghi đè cả 5 cột bằng giá trị từ body, kể cả khi field không được gửi (undefined → NULL)

Kết quả thực tế: PUT /api/products/3 chỉ gửi {"name": "..."} → price, description, imageUrl, category_id của sản phẩm đều thành NULL. PUT với body rỗng {} xóa sạch cả 5 trường cùng lúc.

Ảnh hưởng — bất kỳ form sửa sản phẩm nào chỉ gửi các trường người dùng thực sự chỉnh (hành vi thông thường của UI) đều vô tình xóa sạch dữ liệu các trường còn lại — và trên sản phẩm id chẵn, còn trực tiếp gây ra BUG-19.

BUG-21 — `PUT / DELETE /api/products/:id` không yêu cầu xác thực

API	API-03
Mức độ	Critical
Đặc tả bị vi phạm	SEC-02 + FR-15: "Admin có thể Thêm/Sửa/Xóa sản phẩm"
Test case	TC-PRODUPD-027, 028, 030, 102
Đối chứng	TC-PRODUPD-035 (<code>PUT /api/categories/:id</code> — route "anh em" cùng nhóm admin — CÓ <code>authenticateToken</code> , xác nhận đây là thiếu sót không nhất quán, không phải chủ đích)

`PUT` và `DELETE` trên sản phẩm thành công với **không có header, token rác, hay bất kỳ trạng thái xác thực nào** — route hoàn toàn không có middleware `authenticateToken` , khác hẳn route `/api/categories/:id` cùng nhóm chức năng.

BUG-22 — `PUT /api/products/:id` không kiểm role admin (SEC-03)

API	API-03
Mức độ	Critical
Test case	TC-PRODUPD-029

(Về lý thuyết đây là lớp bảo vệ thứ 2 sau BUG-21; vì BUG-21 đã bỏ hẳn xác thực nên lớp role-check càng không tồn tại — ghi riêng vì đây là 2 lớp phòng thủ độc lập theo đúng SEC-02/SEC-03.)

BUG-23 — Không validate `name / price / category_id` theo FR-15

API	API-03
Mức độ	High
Test case	TC-PRODUPD-002, 004, 006, 007, 008, 009, 010, 040

Tổng hợp 8 test case: `name` rỗng, `name` 256 ký tự (vượt 255), `price` = 0 , `price` âm, `price` sai kiểu (chuỗi chữ), `category_id` không tồn tại, `category_id` thiếu — **tất cả đều được chấp nhận** (200), không một ràng buộc nào trong FR-15 được thực thi ở phía server. `TC-PRODUPD-040` là hệ quả ở tầng schema của chính lỗi này: gửi `name` rỗng lẽ ra phải nhận `400 {error: string}` , nhưng nhận `200` nên assertion kiểm shape của response lỗi đỏ ("*expected undefined to be a string*").

BUG-24 — `:id` sai định dạng/không hợp lệ vẫn xử lý như thành công

API	API-03
Mức độ	Medium
Test case	TC-PRODUPD-015, TC-PRODUPD-031

`:id = "abc"` hoặc chứa payload SQLi-shape (`1 OR 1=1`) đều nhận **200 "Product updated"** thay vì 400 — vì parameterized query chỉ đơn giản không khớp dòng nào (`this.changes = 0`), nhưng handler không kiểm giá trị này trước khi trả về thành công.

BUG-25 — `:id` không tồn tại vẫn báo thành công/trả `200 {}` thay vì 404

API	API-03
Mức độ	Medium
Test case	TC-PRODUPD-014, 024, 025, 042, 043

`PUT / GET` trên sản phẩm không tồn tại (kể cả sau khi `DELETE`) đều không trả 404 — `GET` trả `200 {}` , `PUT` trả `200 {"message":"Product updated"}` dù không có dòng nào bị ảnh hưởng.

BUG-26 — `price` bị ép kiểu thành string khi `id` chẵn

API	API-03
Mức độ	Low
Test case	TC-PRODUPD-039
Vị trí mã nguồn	<code>server.js:162</code>

Độc lập với BUG-19/20 (crash khi giá trị là `null`): ngay cả khi `price` là số hợp lệ, `GET` sản phẩm `id` chẵn luôn trả `price` dạng **chuỗi** (`"100000"`), trong khi `id` lẻ trả **number** (`100000`) — client phải tự xử lý 2 kiểu dữ liệu khác nhau cho cùng một field.

BUG-27 — HTML lỗi lộ stack trace khi body PUT không phải JSON hợp lệ

API	API-03
Mức độ	High
Test case	TC-PRODUPD-045

Cùng nguyên nhân và mức độ với BUG-07 (API-01) — `body-parser` ném lỗi parse, Express trả trang HTML mặc định kèm đường dẫn tuyệt đối của server.

4. Giả thuyết đã bị loại sau khi kiểm chứng

#	Giả thuyết	Đã kiểm bằng gì	Vì sao bị loại
1	(API-02) "Có thể nhảy cóc pending → shipping hoặc pending → delivered "	curl PUT admin/orders/status trực tiếp	Cả hai đều bị chặn đúng — code có kiểm đủ điều kiện transition hợp lệ cho các nhánh này, chỉ riêng canceled → delivered là lỗi hổng thật (BUG-17)
2	(API-02) "user thường huỷ đơn của người khác qua PUT /orders/:id/cancel "	curl với admin_token gọi huỷ đơn của user	404 , không phải lỗi hổng — endpoint này CÓ lọc đúng WHERE user_id = req.user.id , admin gọi vào đơn người khác thì không tìm thấy. Giữ lại làm case đối chứng ở TC-COUPON-105
3	(API-02) "VIP100 cho phép dùng vượt quá 2 lượt/người khi dùng đúng user_id "	Chuỗi 5 request thật (TC- COUPON-102→102e)	Sai — khi user_id đúng và thật, giới hạn 2 lượt hoạt động chính xác (400 ở lượt thứ 3). Lỗi hổng thật nằm ở việc user_id do CLIENT tự khai (BUG-13), không phải ở logic đếm
4	(API-01) "SQL injection qua email bypass được đăng nhập"	3 dạng payload SQLi khác nhau qua curl	Không bypass được — db.get(..., [email], ...) dùng tham số hoá đúng chuẩn (SEC-05 đạt cho endpoint này)

5. Rủi ro / câu hỏi nghiệp vụ (chưa đủ căn cứ gọi là bug)

#	Quan sát	Vì sao chưa gọi là bug
R-01	POST /api/coupon-usage không tự động được gọi bởi POST /api/checkout — phải gọi tay	Có thể là thiết kế cố ý (tách bước ghi nhận usage khỏi checkout để FE tự quyết định khi nào coi là "đã dùng"); cần hỏi lại yêu cầu nghiệp vụ thật trước khi kết luận là thiếu sót
R-02	imageUrl chấp nhận bất kỳ chuỗi nào, kể cả javascript:	Đặc tả không định nghĩa validate URL; rủi ro thật chỉ phát sinh nếu frontend render trực tiếp giá trị này mà không escape (thuộc phạm vi SEC-04 ở tầng UI, ngoài phạm vi 3 API đã chọn của bài này)